

Tema 3 – APIs e Integración de servicios y plataformas – Clase 2

EBIS Business Techschool

Contenidos

- Autenticación con API Keys, Bearer Tokens y variables de entorno
- Consultar una API protegida
- OAuth 2.0 explicado de forma práctica
- Uso de OAuth y renovación de tokens



Autenticación con API Keys, Bearer Tokens y variables de entorno



EBIS Business Techschool

Autenticación en APIs

API Key: Identificación básica

Una clave simple que identifica la aplicación que realiza solicitudes.
Se usa principalmente para monitoreo y control de uso, no para autenticar usuarios.

Bearer Token: Autenticación segura

Un token que verifica la identidad y permisos del usuario o aplicación.
Se usa para controlar acceso y proteger operaciones sensibles.

Qué es una API Key

Identificación del consumidor técnico

Una API Key es una credencial sencilla que identifica a una aplicación, cliente técnico o proyecto. Se usa para controlar cuotas, analítica y facturación, sin identificar necesariamente a una persona.

Cómo se modela en OpenAPI

apiKey en cabecera

Una API Key puede definirse para ser enviada en el encabezado HTTP, lo que permite un control directo y claro en las solicitudes.

apiKey en query string

También es posible transmitir la API Key como parámetro en la URL, aunque esto puede exponerla en registros y caches.

apiKey en cookie

El envío mediante cookies puede ser útil en contextos web, facilitando la gestión automática por parte del navegador.

Por qué conviene usar cabecera para la API Key

Riesgos de enviar la API Key en la URL

Incluir la API Key en la query string aumenta el riesgo de exposición en historiales, registros y herramientas externas.

Ventajas de usar la cabecera X-API-Key

Enviar la API Key en una cabecera personalizada como X-API-Key protege la clave al evitar que aparezca en URLs y registros visibles.

Qué es un Bearer Token

Autenticación de usuario

Un Bearer Token es un token concedido a un usuario autenticado. Normalmente es temporal, se manda en headers y puede tener alcance limitado pero sensible, por lo que su protección en tránsito y almacenamiento es clave.

La idea de 'bearer' en HTTP

La posesión del token es la prueba principal

El token bearer confirma que quien lo presenta está autorizado. Esta confianza hace que el token sea útil y flexible, pero también vulnerable si se filtra o reenvía fuera de canales seguros como HTTPS.

Cómo se envía en la cabecera Authorization

Envío con Bearer Token

El Bearer Token se envía preferentemente en la cabecera Authorization usando el formato *Authorization: Bearer <credencial>*. El servidor puede responder con WWW-Authenticate: Bearer si la credencial falta o no permite acceso.

Formato del token

Bearer indica el método de transporte en HTTP

Un bearer puede ser un JWT o una referencia ofuscada cuya interpretación solo conoce el servidor. En OpenAPI, bearerFormat es una pista documental y no define el tipo ni la seguridad del token.

Opaco y temporal

Token como credencial manejada por el servidor

El token es un identificador temporal que el servidor asocia a una sesión y permisos específicos.

El cliente solo presenta el token sin necesidad de conocer su contenido interno, y el servidor valida su estado y caducidad.

Comparación rápida de API Key y Bearer Token

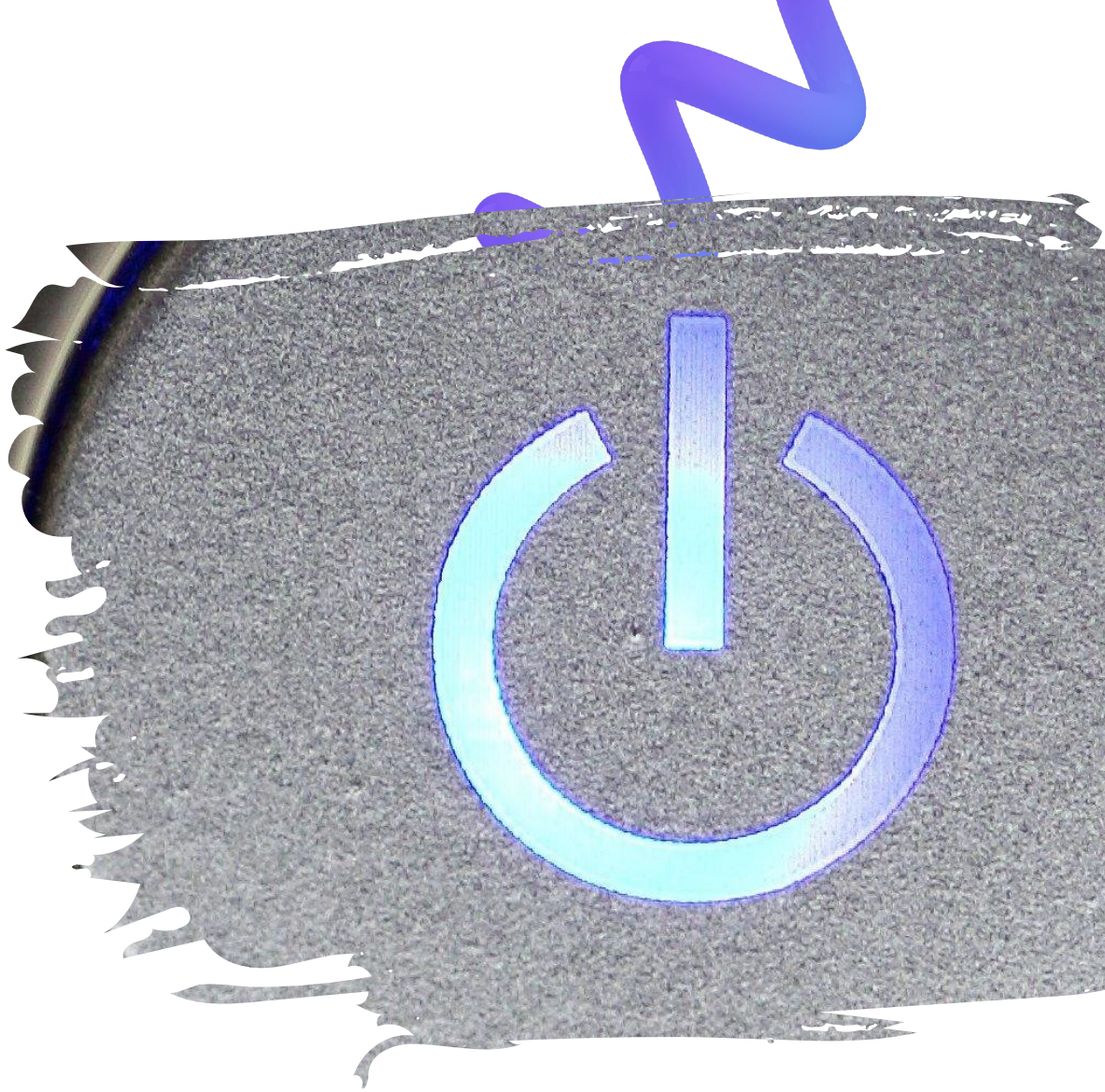
API Key

- Un identificador simple que permite reconocer qué aplicación realiza la petición. Se utiliza principalmente para rastreo y control del uso, ofreciendo un nivel básico de identificación sin autenticación fuerte. No garantiza quién es el usuario final ni permisos detallados, pero ayuda en la monitorización y bloqueo de aplicaciones abusivas.

Bearer Token

- Un token de acceso que representa la identidad del usuario o aplicación que realiza la petición, utilizado típicamente en OAuth para autorización detallada. Permite validar permisos específicos y delega el acceso seguro a recursos, incluyendo autenticación del usuario final. Su uso es esencial para proteger datos sensibles y limitar privilegios.

Cuándo usar una API Key



Identidad técnica simple

Una API Key es adecuada cuando se necesita una forma sencilla de identificar al consumidor técnico sin complejidad adicional. También apoya la gestión de cuotas por cliente y proporciona análisis de uso para facilitar el control y la facturación. Es especialmente útil en portales de desarrolladores para agilizar el acceso y la integración con APIs públicas o semiabiertas.

Cuándo utilizar un Bearer Token

Autorización segura con control y duración limitada

Un Bearer Token es ideal para acceder a recursos protegidos, otorgando permisos específicos según la acción, con autenticación que expira y puede ser revocada para mayor seguridad. Este método es propio para sesiones de usuarios o accesos de servicios con permisos acotados, manteniendo clara la relación entre credencial y contexto de uso.

Riesgos principales de las API Keys

Filtración y uso indebido de API Keys

La exposición de una API Key puede permitir accesos no autorizados en aplicaciones no previstas. Además, el uso sin restricciones y la falta de control aumentan el riesgo de sobrepermisos y violaciones de seguridad.

Riesgos principales de los Bearer Tokens

Exposición y reutilización

El Bearer Token puede ser robado, reenviado o reutilizado si se expone durante la transmisión, en almacenamiento o a través de registros de acceso. Su compromiso incrementa si el token tiene una vida útil larga o permisos demasiado amplios.

HTTPS y TLS siempre

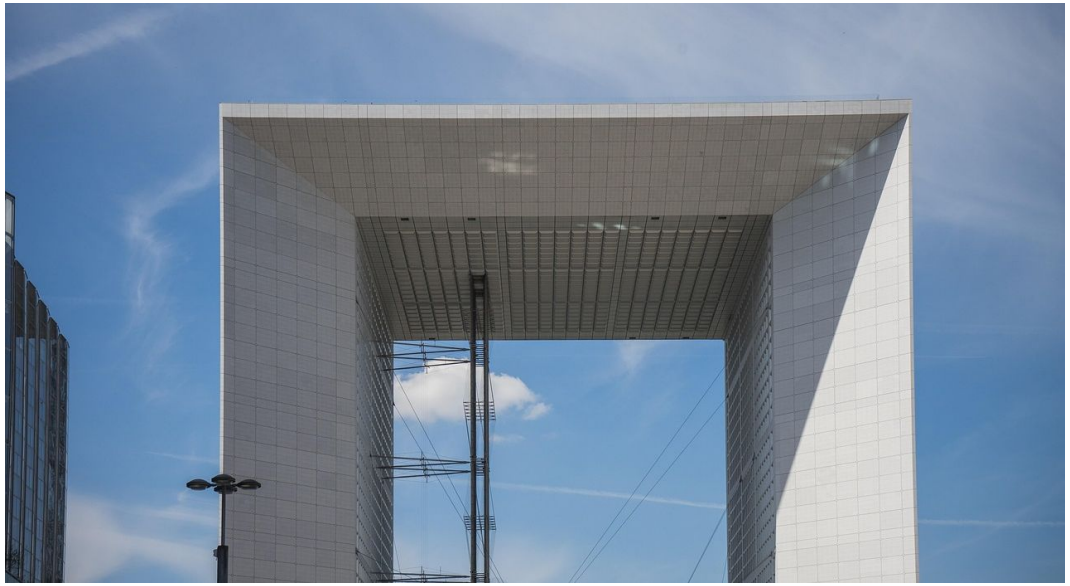
Protección esencial de las credenciales

Toda credencial sensible debe viajar exclusivamente por canales cifrados mediante HTTPS para evitar que sea interceptada o robada en tránsito.

TLS como base irrenunciable

En el caso de los Bearer Tokens, el uso de TLS no es opcional, sino un componente fundamental para garantizar una implementación segura y confiable.

Principio de mínimo privilegio



Scopes mínimos

En la gestión de Bearer Tokens, se utilizan scopes que definen permisos específicos para limitar lo que una credencial puede hacer, garantizando que solo se permita lo necesario.

Restricciones en API Keys

Las API Keys pueden configurarse con restricciones específicas según aplicación, dirección IP, producto o entorno, para evitar que tengan un acceso genérico o excesivo.

Reducción intencionada de poder

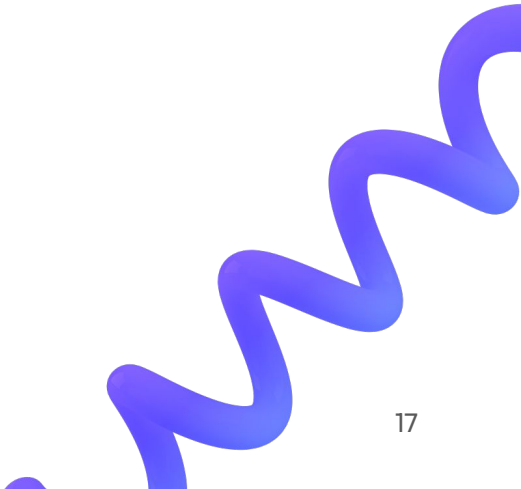
Representar los permisos como partes pequeñas frente a un acceso genérico ayuda a entender visualmente que la seguridad mejora al reducir la capacidad de cada credencial.

Scopes en Bearer Tokens



Limitación de operaciones mediante scopes

Los scopes definen los permisos específicos que un token posee para ejecutar acciones dentro del API. Esto restringe las operaciones permitidas para un consumidor o usuario, evitando accesos no deseados y asegurando que solo realicen acciones autorizadas.



Restricciones para API Keys

Limitaciones por origen y plataforma

Restringir el uso de API Keys según origen, aplicación, plataforma o servicio ayuda a limitar su uso solo a escenarios autorizados, reduciendo riesgos en caso de filtración.

Trazabilidad del consumo

Estas limitaciones permiten identificar y rastrear el consumo de las API Keys por cliente o producto, facilitando la gestión y control operativo.



Rotación y revocación de credenciales

Reducir el tiempo de exposición

Actualizar credenciales periódicamente limita la ventana en que un token comprometido puede ser explotado.

Respuesta rápida ante incidentes

Revocar claves o tokens permite retirar inmediatamente accesos sospechosos o comprometidos.

Continuidad segura y controlada

Mantener un ciclo de renovación continuo asegura accesos válidos sin prolongar la vida útil de las mismas credenciales.

Almacenamiento seguro de secretos

Protección de claves y tokens

Las claves y los tokens deben almacenarse de forma segura para evitar exposiciones accidentales. Nunca se deben incluir en texto claro dentro del código fuente ni en repositorios compartidos.

En el servidor, es preferible guardar representaciones seguras como huellas, hashes o identificadores internos según el tipo de secreto para mantener su confidencialidad.

No registrar secretos en los logs

Diferencia entre trazabilidad segura y exposición de datos

Los logs deben incluir referencias o identificadores parciales para registrar eventos de seguridad sin revelar tokens, contraseñas o claves en texto claro que puedan provocar fugas.

Credenciales fuera de la URL

Riesgos de enviar credenciales en la cadena de consulta

Las credenciales en la URL pueden quedar expuestas en historiales, proxies y registros.
Usar la cabecera HTTP para enviar credenciales es una práctica más segura y confiable.

CORS y cabeceras permitidas

CORS regula el acceso del navegador entre orígenes distintos

CORS no autentica al usuario ni a la aplicación, pero define qué solicitudes están permitidas desde el navegador.

El uso de Authorization o X-API-Key requiere configuración explícita

Para que el navegador envíe estas cabeceras en peticiones con credenciales, el servidor debe indicarlas en Access-Control-Allow-Headers durante el preflight.

Autenticación correcta no garantiza envío sin CORS adecuado

Aunque la API valide credenciales, el navegador bloqueará las peticiones si CORS no autoriza las cabeceras usadas.

Rate limiting y respuesta 429

Protección contra abuso de frecuencia

El rate limiting limita la cantidad de peticiones que un cliente puede realizar en un período para evitar sobrecargas y abusos, incluso con credenciales válidas.

Respuesta 429 Too Many Requests

Cuando se supera la frecuencia permitida, el servidor responde con 429, indicando al cliente que debe esperar antes de hacer nuevas solicitudes.

Permisos insuficientes: 403 y a veces, 404

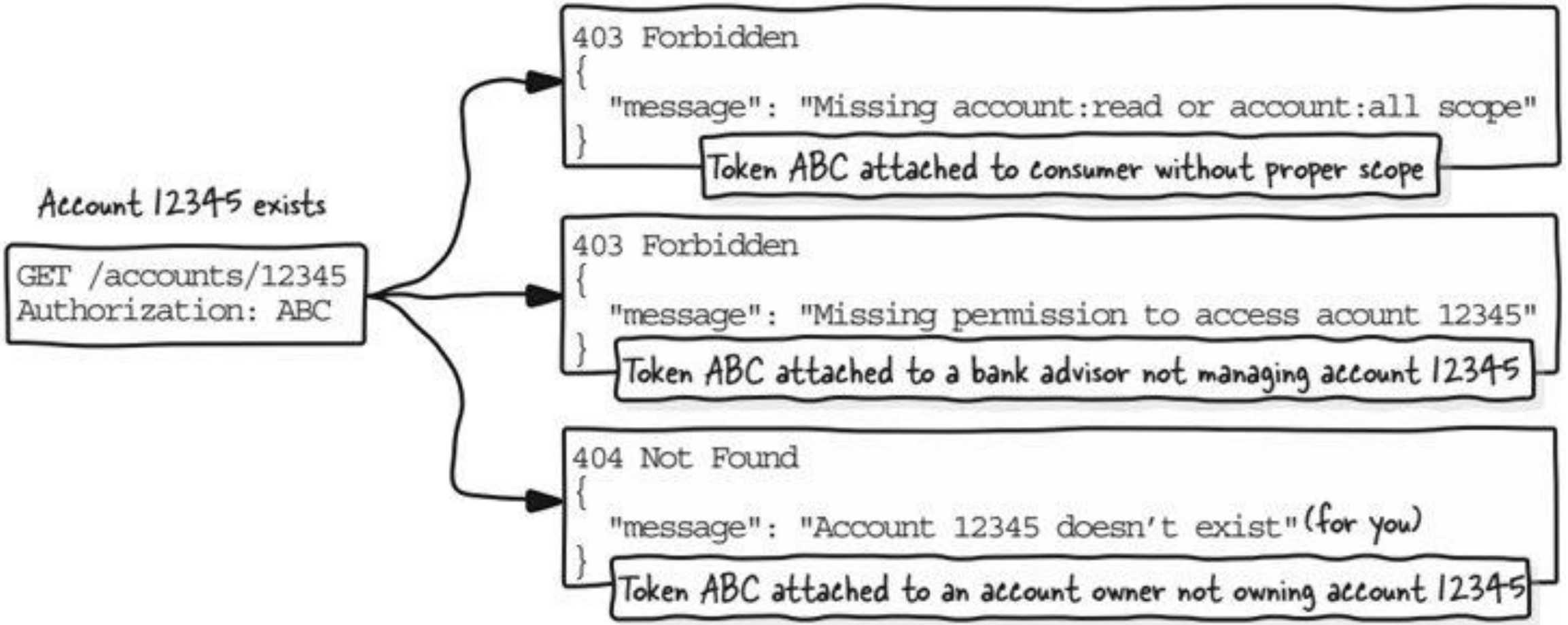
Respuesta 403 Forbidden

Se responde con 403 cuando la credencial es válida pero no autoriza la operación o recurso solicitado. Indica falta de permisos o alcance insuficiente.

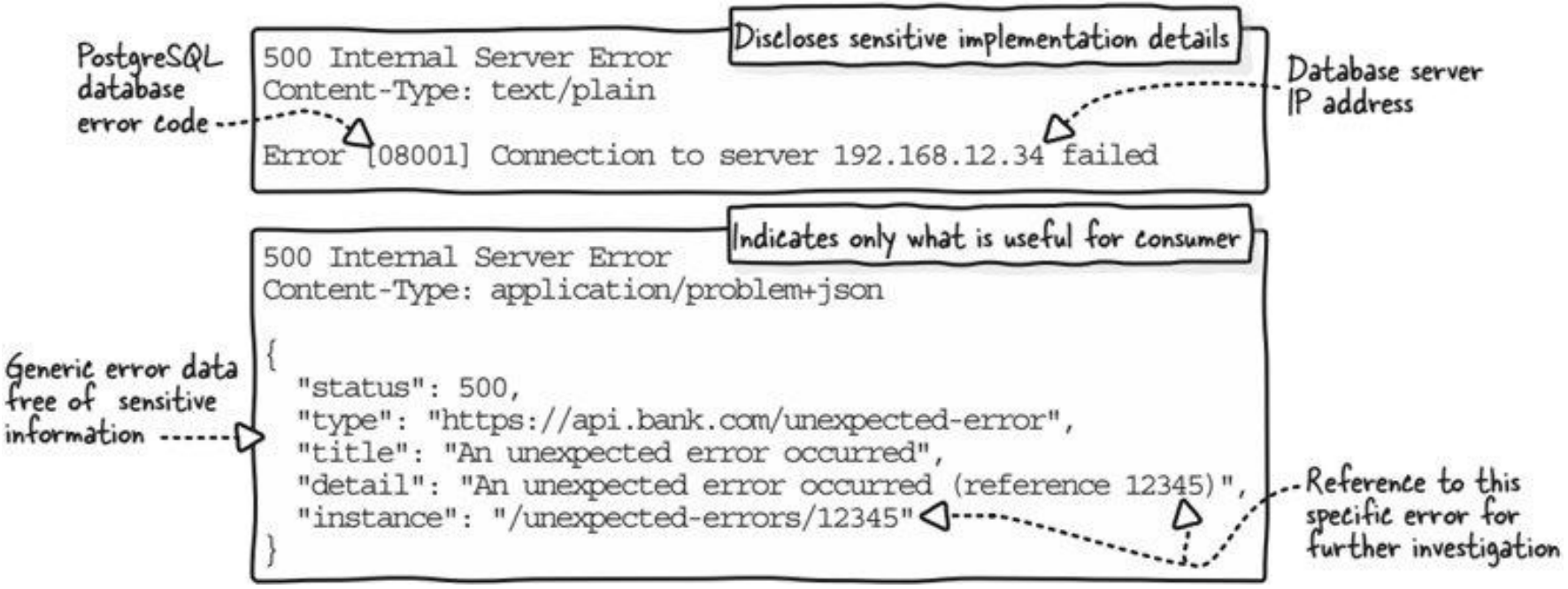
Respuesta 404 Not Found

Se usa 404 para no revelar la existencia del recurso en casos donde ocultar información es importante para la seguridad o privacidad.

Ejemplos


Account 12345 exists
GET /accounts/12345
Authorization: ABC
403 Forbidden
{
"message": "Missing account:read or account:all scope"
}
Token ABC attached to consumer without proper scope
403 Forbidden
{
"message": "Missing permission to access acount 12345"
}
Token ABC attached to a bank advisor not managing account 12345
404 Not Found
{
"message": "Account 12345 doesn't exist" (for you)
}
Token ABC attached to an account owner not owning account 12345

Errores internos: respuesta segura sin revelar detalles



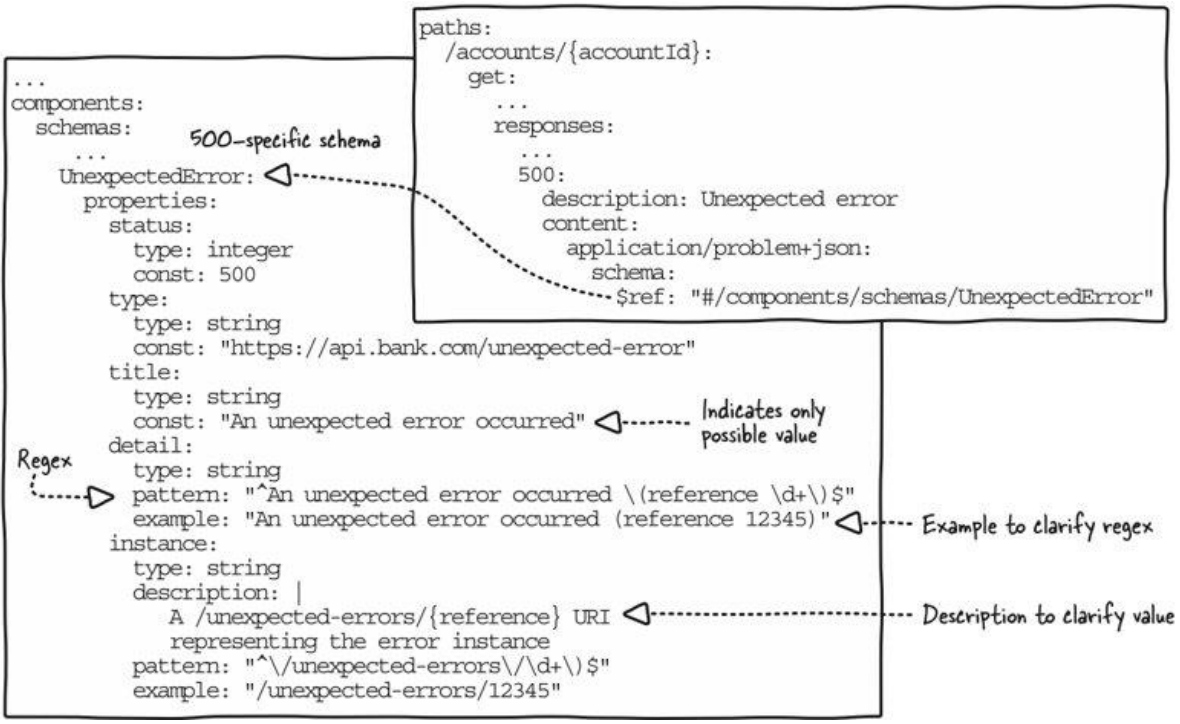
Mensaje genérico que protege la implementación

Los errores internos deben evitar incluir información técnica como nombres de servidores o IPs. Se debe retornar un mensaje claro pero genérico para mantener la seguridad.

Identificador opaco para investigación

Incluir un código o referencia único permite rastrear el error internamente sin exponer datos sensibles al consumidor.

Cómo documentar errores en OpenAPI



Documentación clara de errores

Es fundamental describir de forma precisa en OpenAPI cuándo ocurren errores de autenticación (401), autorización (403, 404), límite de peticiones (429) y errores de servidor (500). Esto permite que la implementación maneje cada caso correctamente y evite improvisaciones.

JSON Schema para respuestas de error

Definición precisa del esquema

Un esquema bien definido para respuestas de error fija el código, el título y la estructura básica del mensaje, evitando formatos ambiguos o datos sensibles inesperados.

Variables de entorno: por qué usarlas

Separación de configuración y código

Permiten evitar incrustar secretos en el código y facilitan cambiar credenciales entre entornos.

Simplifican el despliegue en múltiples entornos sin modificar el código fuente.

Uso de archivos .env en desarrollo

Gestión segura y cómoda de credenciales

Los ficheros .env permiten cargar credenciales y parámetros locales sin incorporarlos al repositorio. Esto mantiene la configuración sensible separada del código compartido, mejorando la ergonomía del equipo.

Gestión centralizada de secretos en producción

Uso de gestores de secretos para credenciales seguras

Las credenciales deben gestionarse mediante un sistema centralizado que reemplace ficheros estáticos o configuraciones vulnerables. Esto permite rotar claves sin interrupciones y mejora la seguridad operativa.

Flujo recomendado con API Key

Emisión controlada

El cliente recibe la API Key de forma segura tras un proceso controlado para autorizar su uso.

Envío en cabecera

El cliente incluye la API Key en la cabecera X-API-Key al hacer cada solicitud a la API.

Validación y respuesta

La API verifica la validez, estado y restricciones de la clave y responde con éxito o errores 401, 403 o 429 según corresponda.

Flujo recomendado con Bearer Token

01

Intercambiar credenciales para emitir token

El cliente envía credenciales al endpoint de emisión para obtener un token seguro que representa su sesión o permiso.

Esto garantiza que sólo usuarios autorizados puedan acceder al sistema.

02

Presentar el token en la cabecera Authorization

Cada petición subsecuente incluye el token en la cabecera Authorization: Bearer para autenticar la solicitud.

Esto permite identificar fácilmente al usuario o servicio autorizado.

03

Validar token, scopes y contexto de usuario

El servidor verifica que el token no haya expirado, que tenga los scopes adecuados y que el contexto de usuario o servicio sea válido.

Así se asegura que sólo se ejecutan acciones permitidas y autorizadas.

Frontends web y la importancia de CORS y almacenamiento seguro

El navegador como entorno de confianza limitada

Al usar credenciales en llamadas a APIs, no basta con autenticar correctamente; es indispensable configurar CORS adecuadamente para controlar el acceso y proteger los secretos.

El frontend debe minimizar la exposición de datos y mantener el tiempo de vida de las credenciales lo más corto posible, anticipando restricciones propias del entorno del navegador.

Integraciones servidor a servidor

Autenticación y autorización seguras sin usuario

En integraciones servidor a servidor, una API Key o un Bearer Token identifican y autorizan aplicaciones automáticamente. Sin embargo, es fundamental limitar permisos al mínimo necesario, rotar credenciales con frecuencia y asegurar el transporte con cifrado para mitigar riesgos en procesos automatizados.

Checklist operativo de seguridad

Uso de HTTPS

Verifica que todas las credenciales y datos sensibles viajen exclusivamente por canales HTTPS para asegurar la confidencialidad.



No exponer datos en URLs

Evita incluir credenciales o información sensible en URLs o parámetros que puedan quedar registrados en logs o historiales.

Control de accesos y límites

Implementa límites de uso, rotación y revocación de credenciales para minimizar riesgos de abuso o accesos no autorizados.

Manejo seguro de errores

Documenta errores sin revelar información sensible para evitar filtraciones y para facilitar debugging seguro.

Decisión entre API Key y Bearer Token

Mecanismo	Identidad Expresada	Nivel de Permiso	Riesgo Asociado	Uso Recomendado
API Key	Identidad app o técnico	Permite gestión básica cuotas	Más vulnerable; fácil de descubrir	Identificar apps y gestionar cuotas sin autorización estricta
Bearer Token	Identidad consumidor y usuario	Control granular de acceso con expiración	Menor riesgo con protocolos seguros (OAuth)	Contexto, autorización fina y seguridad para usuarios finales

Práctica con APIs protegidas



EBIS Business Techschool

Qué API es más segura?:

a) https://clima.com/current?api_key=0a0q3po4th…

b) http://nutricion.com/calorias?plato=sardinas
Headers: {‘x-api-key’: ‘3pq02ht438p…’, ...}

APIs protegidas

- Por parametro de query: https://openweathermap.org/
- Por header: https://api-ninjas.com/api/nutrition
- Por bearer token: https://developers.openai.com/

Ejercicio

- Realiza un script básico para consultar un modelo de OpenAI a través de la API con Python, sin usar la librería de openai

OAuth 2.0 explicado de forma práctica



EBIS Business Techschool

Por qué existe OAuth 2.0

Delegación segura de acceso

OAuth 2.0 permite que una aplicación acceda a recursos protegidos sin pedir al usuario sus credenciales directamente.

Esto reduce el riesgo al evitar compartir contraseñas con terceros y facilita el acceso delegado.

Los cuatro actores del modelo OAuth

01

Resource Owner

Es el usuario que posee los datos o recursos protegidos dentro del sistema.

02

Client

Aplicación o servicio que solicita acceso en nombre del resource owner para usar los recursos.

03

Authorization Server

Se encarga de autenticar al resource owner y emitir tokens de acceso al client autorizado.

04

Resource Server

Servidor que protege y sirve los recursos solicitados, validando los tokens recibidos del client.

Resource owner: quien autoriza

Control y consentimiento

El resource owner es la persona o entidad que controla el recurso protegido.

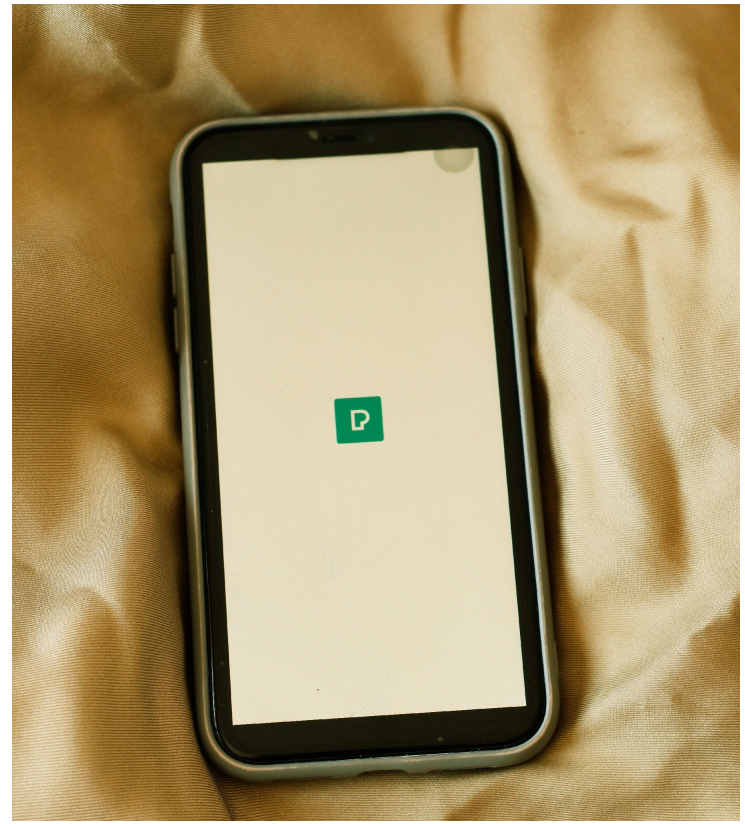
Su autorización depende de su consentimiento, no de la relación directa entre el usuario y la aplicación cliente.

Cliente: la aplicación que solicita acceso

Aplicaciones como clientes

El client es la aplicación que solicita acceso al recurso protegido en nombre del usuario o por su propia identidad.

Puede ser una app web, una app móvil o un backend que actúa como cliente dentro del ecosistema OAuth.



Authorization server: autentica y emite tokens

Función del servidor de autorización

El authorization server autentica al propietario del recurso y obtiene su consentimiento.

Actúa como la autoridad central para emitir y validar tokens de acceso, controlando el permiso delegado de manera segura y confiable.



Resource server: donde viven los recursos protegidos



Alojamiento y validación de tokens

Este servidor aloja los recursos protegidos y acepta solicitudes solo con tokens de acceso válidos.

No autentica directamente al usuario, sino que confía en los tokens emitidos por el authorization server.

La ventaja estructural de separar roles

01

Modelo inseguro: Compartir credenciales

El usuario debe ingresar sus credenciales directamente en el cliente tercero, aumentando el riesgo de exposición y mal uso.

02

Modelo seguro: Separación de roles

Las credenciales se autentican solo en el authorization server, protegiendo la identidad y fortaleciendo el control de acceso delegado.

Patrón general de una arquitectura OAuth

01

Obtener autorización o usar credenciales propias

El cliente obtiene permiso del usuario o emplea sus credenciales para iniciar el proceso.

Esto asegura que la acción esté autorizada y controlada.

02

Intercambiar autorización por un token de acceso

El cliente solicita un access token basado en la autorización recibida.

Este token representa la autorización de forma segura y limitada.

03

Presentar el token al resource server

El cliente utiliza el access token para acceder a recursos protegidos.

Esto evita el uso directo de credenciales del usuario y mantiene la seguridad.

El acceso termina como una llamada con Bearer

Uso del token en autorización

El cliente presenta el access token mediante el encabezado HTTP Authorization.

Para tokens Bearer, esto sigue el estándar RFC 6750 y refleja cómo realmente se consume la API.



Parámetros clave de la authorization request

Parámetros esenciales

Los parámetros principales incluyen response_type=code, client_id y redirect_uri cuando es necesario.

Se recomienda usar el parámetro state para mantener contexto y prevenir ataques CSRF.

Redirección y devolución del código

Proceso de redirección y código

El servidor de autorización redirige el navegador hacia el endpoint de redirección del cliente con un código temporal.

El usuario nunca entrega sus credenciales al cliente, asegurando así la seguridad del proceso.

Intercambio del code por access token

01

Código de Autorización

El cliente envía el código obtenido y, si es necesario, el mismo redirect_uri para validar la petición.

02

Acceso al Token

El endpoint de tokens responde con el access token, completando el proceso de autenticación.

PKCE: protección frente a la interceptación del código

Extensión de seguridad adicional

PKCE ata el código de autorización a la instancia que inicia la transacción.
Esto reduce el riesgo de que un código sea robado durante su transmisión.



code_verifier y code_challenge

01

Generación del code_verifier

El cliente crea un code_verifier aleatorio y seguro como base para el proceso de autenticación.

Este valor es privado y no se comparte inicialmente con el servidor.

02

Uso del code_challenge

El cliente deriva un code_challenge del code_verifier y lo envía al servidor.

El servidor recibe solo este desafío y lo utiliza para validar la autenticación posteriormente.



Authorization Code con PKCE, de extremo a extremo

01

Generación segura

Se crean el state y el code_verifier para proteger la sesión y evitar ataques de reutilización.

02

Solicitud y redirección

El usuario inicia la autorización y el servidor redirige con el código de autorización.

03

Canje por token

El backend intercambia el código usando PKCE para obtener el access token de forma segura.

Qué se ve desde fuera: siempre aparece Bearer

La autenticación en la práctica

Aunque OAuth describe un proceso complejo, al final el token de acceso se envía en la cabecera Authorization como "Bearer <token>". Esto hace que, para un observador externo, la llamada a la API se perciba simplemente como una solicitud con un encabezado Authorization que incluye una credencial.

ROPc: qué hacía el Resource Owner Password Credentials

Flujo histórico con credenciales directas

El flujo ROPC implica que el usuario entrega sus credenciales directamente al cliente.

Esto permite que el cliente gestione el acceso pero sacrifica la separación de responsabilidades y la seguridad, ya que el cliente maneja la contraseña del usuario.

Por qué ROPC no debe usarse

01

Exposición de credenciales

ROPC envía las credenciales del usuario directamente al cliente, lo que aumenta el riesgo de comprometerlas.

02

Compatibilidad limitada

Este flujo no funciona bien con autenticación multifactor ni otros mecanismos modernos de seguridad.

03

No cumple estándares actuales

RFC 9700 establece que ROPC ya no satisface las expectativas de seguridad contemporáneas y debe evitarse.

Casos de uso del acceso máquina a máquina

01

Procesos batch

Automatización de tareas repetitivas que se ejecutan sin intervención humana directa.

02

Integración entre microservicios

Comunicación segura y continua entre servicios dentro de una arquitectura distribuida.

03

Sincronización de datos

Actualización periódica y automática de información entre sistemas internos.

04

APIs internas

Acceso controlado y sin usuario visible para llamadas a API dentro de la empresa.

La seguridad moderna de OAuth 2.0



Enfoque multifacético

La seguridad actual no depende de un solo mecanismo.

Se basa en la combinación coherente de Authorization Code con PKCE, rechazo del implicit, rechazo de ROPC y precauciones en el transporte de tokens.

Errores comunes de implementación

01

Falta de validación del estado

No validar el parámetro state expone la aplicación a ataques de falsificación de solicitudes entre sitios, poniendo en riesgo la seguridad del usuario.

02

Uso incorrecto de redirect_uri

Reutilizar o no restringir el redirect_uri puede llevar a redirecciones inseguras y capturas de tokens por actores maliciosos.

03

Ignorar PKCE y exposición de tokens

No implementar PKCE deja vulnerables los flujos públicos y almacenar tokens en lugares inseguros aumenta el riesgo de filtraciones.

Ideas clave para llevarse

Separación de actores

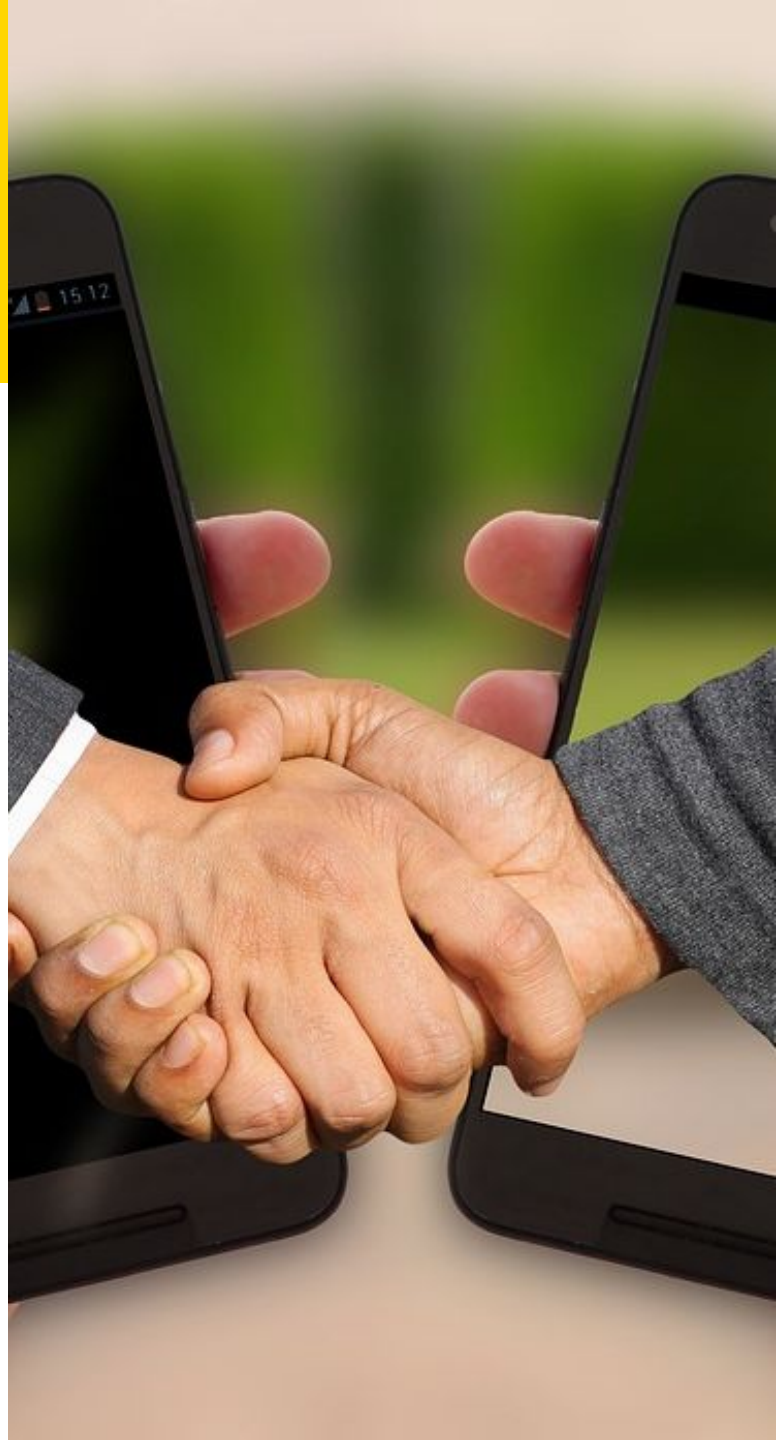
OAuth 2.0 distingue claramente entre usuarios, clientes y servidores de autorización para mayor seguridad.

Flujos recomendados

Authorization Code con PKCE es ideal para usuarios; Client Credentials para comunicación entre servicios.

Flujos obsoletos

Evite utilizar flujos antiguos que no cumplen con estándares modernos de seguridad.



Práctica con OAuth



EBIS Business Techschool

OAuth Linkedin

- **API con OAuth:**
  https://www.linkedin.com/developers/apps/228683010/auth



EBIS Business Techschool

www.ebiseducation.com